



06 — Retrieval Pipeline

Overview

The retrieval pipeline combines two complementary mechanisms: **direct cosine similarity** (exact FAISS search) and **spreading activation** over the prototype graph (pattern completion). Results are scored, merged, temporally boosted, and salience-re-ranked.

Mathematical Formulation

Phase 1: Query Encoding

$$\mathbf{q} = \text{encode}(\text{query}) \in \mathbb{R}^d, \quad \|\mathbf{q}\|_2 = 1$$

Phase 2: Cosine-Direct Retrieval

FAISS search over episodic embeddings:

$$\mathcal{C} = \{(i, s_i) \mid i \in \text{FAISS.search}(\mathbf{q}, k_e), s_i = \langle \mathbf{q}, \mathbf{e}_i \rangle\}$$

Where $k_e = \text{episodic_top_k}$ (default: 10).

Prototype search:

$$\mathcal{P}_{\text{seed}} = \{(j, s_j) \mid j \in \text{SemanticStore.search}(\mathbf{q}, k_s), s_j = \langle \mathbf{q}, \mathbf{c}_j \rangle\}$$

Where $k_s = \text{semantic_top_k}$ (default: 6).

Phase 3: Spreading Activation (Stage 1)

When `use_spreading = True`, activation propagates from seed prototypes over the graph.

Initialization: seed prototypes receive activation proportional to their cosine match:

$$a_0(p) = \frac{\max(0, \langle \mathbf{q}, \mathbf{c}_p \rangle)}{\max_j \langle \mathbf{q}, \mathbf{c}_j \rangle}$$

Propagation (for `spread_steps` iterations):

$$a_{t+1}(p) = \alpha \cdot a_t(p) + (1 - \alpha) \sum_{q \in \mathcal{N}_{\text{in}}(p)} \frac{w_{qp}}{\sum_r w_{qr}} \cdot a_t(q)$$

Where:

- $\alpha = \text{spread_decay}$ (default: `0.6`) – retention factor
- $\mathcal{N}_{\text{in}}(p) = \text{prototypes with outgoing edges to } p$
- $w_{qp} = \text{stored edge weight (fused similarity + transition + coactivation)}$

Pruning: below `spread_activation_floor` (default: `10^{-3}`), activation is zeroed each step.

Salience gate (optional): final activation modulated by prototype support:

$$a_{\text{final}}(p) = a(p) \cdot (1 + 0.1 \cdot \sqrt{1 + \text{support_count}(p)})$$

Phase 4: Episode Harvesting

From each activated prototype, harvest member episodes:

$$\mathcal{H} = \bigcup_{p : a(p) > 0} \{(e, a(p) \cdot w_{\text{spread}})\}$$

Where $w_{\text{spread}} = \text{spread_episode_weight}$ (default: `0.15`).

Score ceiling: graph-harvested scores are capped at:

$$s_{\text{harvest}} \leq \gamma_{\text{ceiling}} \cdot \min_{e \in \mathcal{C}} s_e$$

Where $\gamma_{\text{ceiling}} = \text{spread_score_ceiling}$ (default: `0.9`).

Diversity cap: at most `diversity_per_prototype` (default: `2`) episodes per prototype from graph harvest.

Phase 5: Temporal Boost (Stage 7)

For each candidate episode e with timestamp t_e :

$$s_{\text{temporal}}(e) = \alpha_t \cdot \cos(\mathbf{t}_q, \mathbf{t}_e)$$

Where:

- $\mathbf{t}_q$ = temporal embedding of the query anchor timestamp
- $\mathbf{t}_e$ = temporal embedding of the episode timestamp
- α_t = `temporal_weight` (default: `0.25`)

Temporal embeddings are multi-scale sinusoidal (see §07 — Temporal Context).

Phase 6: Saliency Re-Ranking

Final score for each episode:

$$s_{\text{final}}(e) = \underbrace{\cos(\mathbf{q}, \mathbf{e})}_{\text{semantic}} + \underbrace{\alpha_t \cdot \cos(\mathbf{t}_q, \mathbf{t}_e)}_{\text{temporal}} + \underbrace{\alpha_s \cdot s_{\text{saliency}}(e)}_{\text{saliency}}$$

Where α_s = `saliency_weight` (default: `0.3`).

For graph-harvested episodes, the semantic component is replaced by the harvest score (capped).

Phase 7: Reinforcement

Only cosine-direct episodes in the top slice receive recall reinforcement (+ `saliency_weight` to saliency, +1 to `recalled_count`). Graph-harvested episodes do not receive reinforcement to prevent self-rewarding feedback loops.

Configuration

RetrievalConfig

Parameter	Type	Default	Description
<code>episodic_top_k</code>	<code>int</code>	<code>10</code>	Cosine-direct episodes to fetch
<code>semantic_top_k</code>	<code>int</code>	<code>6</code>	Cosine-direct prototypes to fetch
<code>neighbor_top_k</code>	<code>int</code>	<code>6</code>	Neighbors per prototype in

Parameter	Type	Default	Description
			propagation
<code>salience_weight</code> α_s	float	0.3	Salience contribution to final score
<code>use_spreading</code>	bool	True	Enable spreading activation
<code>spread_steps</code>	int	2	Propagation iterations
<code>spread_decay</code> α	float	0.6	Activation retention per step
<code>spread_activation_floor</code>	float	10^{-3}	Pruning threshold
<code>episodes_per_prototype</code>	int	6	Episodes harvested per prototype
<code>spread_episode_weight</code>	float	0.15	Base score for graph-harvested episodes
<code>spread_score_ceiling</code>	float	0.9	Max harvest score relative to cosine
<code>salience_gate</code>	bool	True	Enable support-count modulation
<code>diversity_per_prototype</code>	int	2	Max graph episodes per prototype
<code>use_temporal</code>	bool	True	Enable temporal proximity bonus
<code>temporal_weight</code> α_t	float	0.25	Temporal contribution to final score

Key Invariants

1. Cosine-direct episodes always outrank graph-harvested ones (score ceiling + low base weight).
2. Spreading activation solves pattern completion: episodes with weak cosine matches can still be retrieved if reachable through high-weight edges.
3. Only cosine-direct episodes receive reinforcement — prevents graph self-reinforcement.
4. Temporal boost is additive, not multiplicative — it nudges ranking rather than dominating.
5. Salience gate modestly boosts well-consolidated prototypes (Hebbian: "fire together, wire together").